Response to reviewers for TCC-2025-12-0666

Dear Professor Guo and Reviewers,

Thank you for reviewing our manuscript. We have revised it under the new title, "Closed-Loop Threat-Informed Remediation of Cloud-Native Kubernetes Security Misconfigurations." The earlier submission did not explain the contribution, evidence boundary, or limitations clearly enough. It also contained reference problems that should have been caught before submission. We corrected those problems and narrowed several claims.

For each point below, we state what changed and where it appears in the revised manuscript. We refer to sections, figures, and tables because line numbers may change during final typesetting.

Response to Reviewer 1

We appreciate the reviewer's recognition that the paper addresses a practical problem, provides a working prototype, includes extensive evaluation, and releases reproducible artifacts.

R1.1 Writing quality, unverifiable references, and the appearance of AI-written text

Reviewer comment: The paper was not written well, appeared partly generated by AI, and contained three references that could not be found.

Response: We agree that this was a serious problem. We removed the unverifiable entries and replaced them with sources whose titles, authors, venues, and links could be checked. We also rechecked the current citation graph. Every citation key used in the manuscript resolves to a bibliography entry, and the removed placeholder keys no longer appear.

We rewrote the manuscript in a conventional research-paper structure and removed terse, product-manual language. The Introduction now states the problem, approach, contributions, and paper organization in full sentences. The design, evaluation, related-work, and limitations sections use the same terminology for the verifier gates and evidence scope.

Manuscript locations: Section 1, "Introduction"; Section 2, "Task and System Model"; Section 7, "Related Work"; and the References.

R1.2 Missing scientific state of the art

Reviewer comment: The paper focused on industrial tools, did not compare adequately with scientific work, and cited academic references that could not be verified.

Response: We expanded Related Work and organized it into five groups: detection-only tooling, admission-time policy engines, LLM-based configuration repair, large-scale SRE automation, and emerging security agents. The revision now discusses LLMSecConfig, Kubecurity, KubeGuard, GenKubeSec, KubeIntellect, KubeLLM, Codex Security, Kyverno, Polaris, and Kubernetes admission mechanisms, along with relevant automated-program-repair research.

We also changed the comparison language. The revised paper does not claim that our system is universally superior. Table 8 compares lifecycle and evidence contracts. Table 9 reports bounded per-policy coverage with the denominators shown and states that it is not a leaderboard.

Manuscript locations: Section 7, "Related Work"; Table 8, "Lifecycle comparison of Kubernetes remediation systems"; and Table 9, "Per-policy coverage on shared security-context slices."

R1.3 Unclear contribution and unclear role of threats

Reviewer comment: The contribution was not stated clearly, and the paper did not explain how threats drive the tool.

Response: We narrowed the contribution. The paper does not claim novelty for scanning Kubernetes manifests, generating patches, or closing a repair loop by itself. The contribution studied here is a proposer-independent acceptance contract for repair candidates. A candidate can carry four forms of evidence: trigger-policy re-check, RFC 6902 patch applicability, Kubernetes

server-side API admission, and cross-policy safety checks. The released campaigns identify which gates they required. The fixture-seeded live replay requires all four.

The revised threat-informed framing has two parts. First, the verifier requires evidence that the triggering security condition was removed without violating the checked safety invariants. Second, the scheduler uses policy risk, historical verifier pass priors, configured service-time priors, and configured exploit-catalog-style boosts to order accepted work. The evaluation describes this as static risk-priority ordering, not online learning.

Manuscript locations: Section 1, "Introduction"; Section 2, "Task and System Model"; Section 3.1, "Pipeline and Verifier Gates"; and Figure 1.

R1.4 Weak design description and disconnected notation

Reviewer comment: The design description was poor, formal variables were not integrated into the explanation, and the comparison interrupted the design discussion.

Response: Section 2 now defines the manifest, detection, patch, and acceptance predicate before Section 3 uses them. It also states the threat model, explains how false positives and detector disagreement are handled, and defines an "attempt" and its retry budget. Section 3 then follows the actual pipeline from detector to proposer, verifier, and scheduler. Figure 1 shows the control flow and the four verifier gates. Two end-to-end examples show how a patch moves through those gates and into the queue.

The broad system comparison was moved to Related Work. The short comparison in Section 3 is limited to the design choice being explained.

Manuscript locations: Sections 2 and 3; Figure 1; and the two walkthroughs in Section 3.2.

R1.5 Introduction, organization, and readability

Reviewer comment: The introduction could be improved, the organization was poor, and the paper required too much effort to read.

Response: The paper now follows the order Introduction, Task and System Model, System Design, Implementation, Evaluation, Ablations and Failure Taxonomy, Related Work, Limitations, and Conclusion. The Introduction gives a short results preview only in the abstract; detailed denominators and campaign results appear in the Evaluation. Low-level operational material was removed from the main narrative or left in the artifact documentation.

Manuscript locations: Sections 1 through 9 and Table 2, which defines the unit and denominator for each headline metric.

Response to Reviewer 2

We appreciate the reviewer's assessment that the topic is relevant, the implementation is technically sound, and the artifact is useful.

R2.1 Introduction structure

Reviewer comment: The introduction should separate the objective, motivation, justification, and problem description.

Response: We kept one standard Introduction section but divided its argument into labeled paragraphs for Problem, Approach, Contributions, and Organization. This keeps the paper's objective separate from the problem statement and moves detailed experimental evidence to the Evaluation.

Manuscript location: Section 1, "Introduction."

R2.2 Choice of tools and technologies

Reviewer comment: The paper should explain why it chose particular tools and technologies and what alternatives were considered.

Response: The revised design explains why RFC 6902 JSON Patch is the candidate artifact, why deterministic rules are the default proposer for covered policy families, why LLM output remains optional, and why Kubernetes server-side dry-run is used for API-level admission evidence. Related Work compares these choices with scanner-only tools, admission-time mutation, iterative LLM repair, schema-guided repair, and general security or Kubernetes agents.

Manuscript locations: Sections 3.1, 4.2, and 7; Tables 1, 8, and 9.

R2.3 Figures and tables were separated from their discussion

Reviewer comment: Some figures and tables were far from the text that cited and explained them.

Response: We reorganized the surrounding discussion and captions so that each visual has a specific role. Figure 1 appears with the system design. Figures 2 and 3 appear with the scheduler and matched-mode results. Tables 4 through 7 appear with the evaluation and ablation discussion. Tables 8 and 9 appear with Related Work and are explained before the five-category survey. We inspected the complete 13-page PDF after the final hosted build for float placement, clipping, and broken references.

Manuscript locations: Sections 3, 5, 6, and 7.

R2.4 Architecture detail and evaluated versions

Reviewer comment: A more detailed architecture figure, including versions of the evaluated elements, would help readers understand the system.

Response: Figure 1 now identifies the detector, proposer, four-part verifier, scheduler, and risk inputs. Section 3 explains the contract between the stages. We did not place version numbers inside the diagram because its purpose is to show control flow. The artifact-availability paragraph records the evaluated environment: Python 3.12.4, kubectl 1.34.1, kube-linter 0.7.6, kind 0.30.0, and Kubernetes 1.34.0. Section 4.2 records the optional Grok/xAI configuration, including model, temperature, seed, timeout, and retry count.

Manuscript locations: Figure 1; Sections 3.1 and 4.2; and Section 9, "Artifact availability."

R2.5 Experimental and LLM limitations

Reviewer comment: The paper should discuss the limitations of the experiments and LLM technology in more detail.

Response: Section 8 now separates external validity, detector validity, fixture sensitivity, provider and LLM risk, verification scope, scheduler validity, human-review boundaries, and experimental validity. Table 10 distinguishes completed campaigns, deterministic replays, and planned work. The manuscript states that the live replay is fixture-seeded, the offline campaigns did not all require a live API gate, the scheduler result is a static finite-queue replay, and no experiment establishes full workload semantic equivalence or completed human-operator validation.

Manuscript locations: Section 8, "Limitations," and Table 10, "Evidence status."

Response to Reviewer 3

We appreciate the reviewer's recognition that the topic is relevant to TCC, that the artifact is reproducible, and that the system appears technically sound. The revision directly addresses the concerns about novelty, comparability, organization, and design detail.

R3.1 Weak novelty and a straightforward combination of existing tools

Reviewer comment: The pipeline appeared to be a straightforward combination of existing tools, with no clear research challenge or contribution suitable for TCC.

Response: We removed broad superiority claims and no longer present a closed repair loop by itself as the novelty. Existing systems already implement useful detect, repair, mutate, validate, or re-scan loops. Our narrower research question is whether heterogeneous patch proposers can be evaluated under one explicit Kubernetes-specific acceptance contract, with each released campaign identifying the gates it actually required.

Table 7 tests the acceptance boundary on matched 5,000-record verifier snapshots. Weaker predicates admit 312 rules-mode records and 551 Grok/xAI policy-only records that the corresponding full-verifier snapshots reject. We state the limits of this evidence: the calculation uses checked-in historical records, not a fresh execution of the current verifier, and it is not evidence about a named competing agent.

Manuscript locations: Sections 1, 3, 6.1, and 7; Table 7.

R3.2 Missing experimental comparison with recent agent-based systems

Reviewer comment: The evaluation did not compare experimentally with systems such as Codex Security or KubeIntellect, so claims of superiority were not supported.

Response: This point is only partially addressed. We did not complete a matched head-to-head experiment. The published systems use different tasks and evaluation interfaces: Codex Security scans connected GitHub repositories, while KubeIntellect evaluates natural-language Kubernetes-management queries. Neither publication reports results under our identical-manifest remediation protocol. Building and validating adapters plus a shared acceptance contract was beyond this revision, and presenting unlike evaluations as a head-to-head result would be misleading. We therefore do not assign our ablation results to those systems and do not claim experimental superiority over them.

The revision adds two bounded forms of comparison. Table 8 compares published lifecycle and evidence contracts. Table 7 holds candidate records fixed and tests weaker versus full acceptance predicates. A named-agent experiment remains future work and is listed as a limitation.

Manuscript locations: Sections 6.1, 7, 8, and 9; Tables 7 and 8.

R3.3 Nonstandard presentation and premature results in the introduction

Reviewer comment: Section 1 should be a conventional Introduction, detailed results should not be presented before the approach, and the paper should state its organization.

Response: Section 1 is now "Introduction" and follows the order problem, approach, contributions, and organization. The detailed results, denominators, confidence intervals, and failure analysis are in Sections 5 and 6. The abstract retains a short quantitative summary, as is customary, while the Introduction does not reproduce the evaluation tables.

Manuscript location: Section 1, followed by Sections 5 and 6 for the detailed results.

R3.4 Original Tables 1 and 2 compared different systems without explanation

Reviewer comment: The two comparison tables used different system rosters and were difficult to interpret.

Response: These comparisons are now Tables 8 and 9. The text explains why their rosters differ. Table 8 compares the closest repair and validation contracts. Table 9 is limited to tools with reusable policy-level artifacts or scanner interfaces for shared slices. The caption and surrounding text state that the denominators differ and that Table 9 is coverage evidence, not an aggregate leaderboard.

Manuscript location: Section 7; Tables 8 and 9.

R3.5 Related Work was not systematic and omitted systems used in the tables

Reviewer comment: Related Work should categorize the field, explain where prior approaches fall short, and discuss systems such as Polaris and LLMSecConfig.

Response: Related Work now uses five categories and discusses every system that appears in the lifecycle comparison. Polaris and LLMSecConfig are included in the text and tables. The section also discusses Kubecurity, KubeGuard, GenKubeSec, Kyverno, KubeIntellect, KubeLLM, Codex Security, OPA Gatekeeper, and relevant repair literature. For each category, we state the comparison boundary without treating unlike metrics as a single ranking.

Manuscript location: Section 7; Tables 8 and 9.

R3.6 False positives, attempts, notation, and research-question rationale

Reviewer comment: The design did not explain false positives or attempts, the notation was not formal enough, and the research questions and findings were difficult to follow.

Response: Section 2 defines a manifest, detection, patch, and acceptance predicate. It explains detector disagreement and false-positive handling: a guarded deterministic proposer emits no edit when the required field is absent, and the policy re-check rejects a patch that does not clear the triggering condition. It also defines an attempt as one proposer-to-verifier cycle and states the retry budget.

Section 5.1 now gives each research question a rationale and a separate finding. Table 2 identifies the unit and denominator for every headline metric so that record-level, patch-level, and risk-mapped results are not mixed.

Manuscript locations: Sections 2.1 and 5.1; Table 2.

R3.7 Implementation section read like a product manual

Reviewer comment: The implementation section was too long and contained reference-manual detail.

Response: The Implementation is now a short Section 4 covering software structure, detection and test evidence, and dataset configuration. Command catalogs, detailed artifact paths, and operational instructions remain in the public artifact rather than the main paper. The main text keeps only the details needed to understand or reproduce the evaluation.

Manuscript location: Section 4 and the separately released artifact.

R3.8 Supplemental material

Reviewer comment: The review form indicated that no supplemental material was available to Reviewer 3.

Response: The revised paper points to an immutable artifact commit and release tag containing the source, evaluation scripts, checked datasets, telemetry, figures, and reproducibility entry point. The resubmission packet will include the supplemental material required by the portal, and we will verify the exact uploaded files before submission.

Manuscript location: Section 9, "Artifact availability."

Consistency of the revised results

The revised text uses the following values throughout:

- The full deterministic rules and guardrails run accepts 13,589 of 13,656

attempted patches, or 99.51 percent. The auto-fix rate is 0.8646 over 15,718 detections, the median patch size is 8 operations, and there are 67 rejected records.

- The fixture-seeded live replay records 1,000 of 1,000 accepted patches passing

both server-side dry-run and live apply.

- The matched Grok/xAI snapshot accepts 4,426 of 5,000 records, or 88.52 percent.
- In the matched finite-queue replay, static risk-priority ordering reduces the

top-50 P95 wait from 147.2 hours under FIFO to 7.8 hours under the shared 10-minute service-time fallback.

These values appear in Sections 5 and 6 and in Tables 2 through 7. The paper does not interpret offline acceptance as universal API admissibility, static queue replay as a live operator study, or snapshot ablation as a named-agent benchmark.

Remaining limitations

The revision does not claim to resolve the following limitations:

- No matched head-to-head experiment was completed against Codex Security or

KubeIntellect.

- No campaign proves full workload semantic equivalence after a patch.
- Historical offline campaigns did not all require a live Kubernetes API gate.
- The scheduling experiment is a static finite-queue replay with configured

service-time priors, not an online bandit or open-arrival fairness study.

- The human-in-the-loop rotation remains future work.

The revision now states what each campaign demonstrates and what it does not. We thank the reviewers for identifying the weaknesses in the earlier version.

Sincerely,

Brian Mendonca and Vijay K. Madisetti

Georgia Institute of Technology